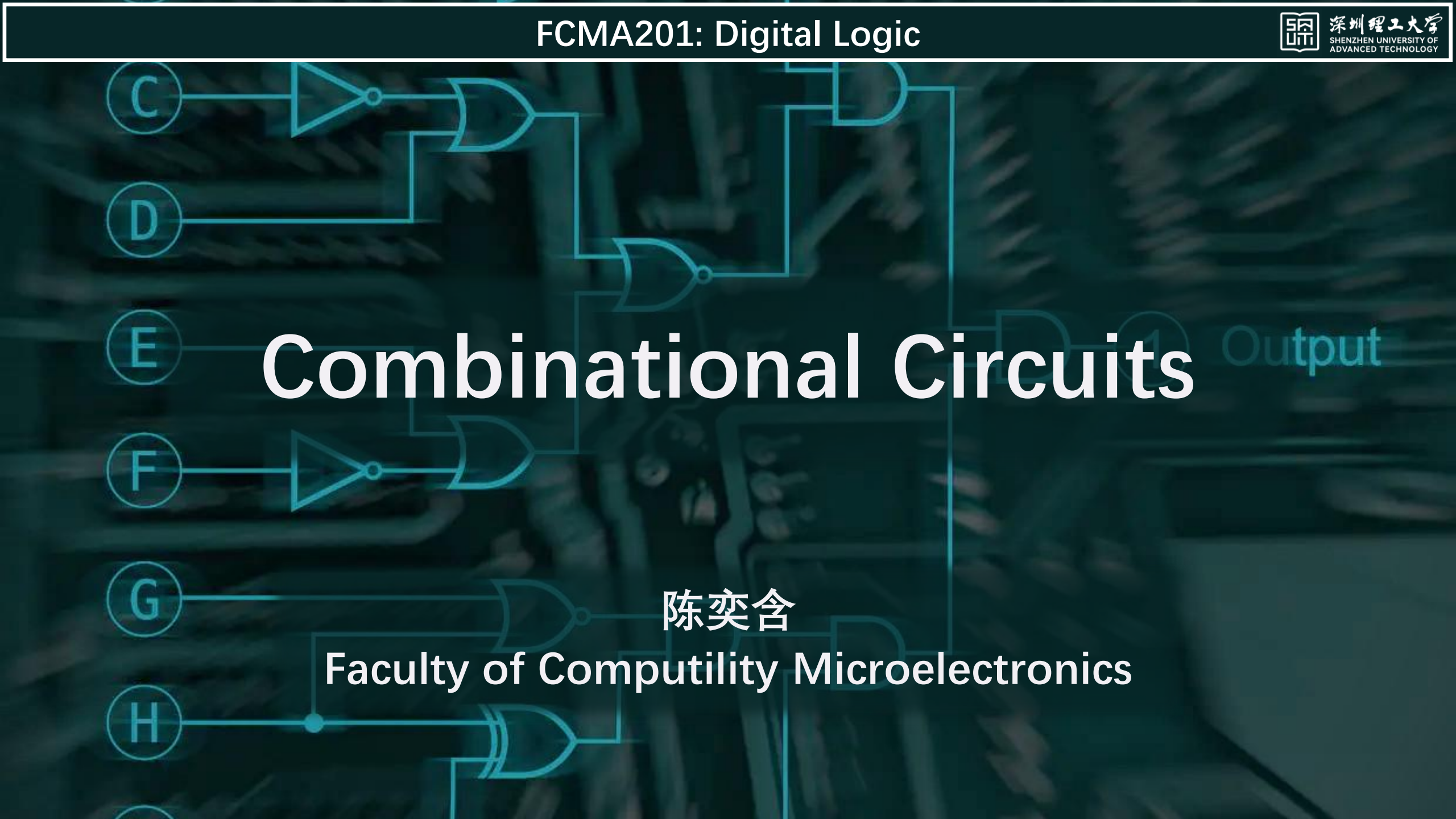


Combinational Circuits

陈奕含
Faculty of Computility Microelectronics

Outline

- **Adders**
  - Half-adder
  - Full-adder
  - Ripple-carry Adder
  - Adder/subtractor Unit
  - Carry-lookahead Adder
  - Hierarchical Design for Large Adder
- **Multiplier**
  - Multiplication for Unsigned Numbers
  - Array Multiplier
  - Multiplication for Signed Numbers
- **Multiplexers**
  - Circuit of Multiplexer
  - Hierarchical Multiplexer
  - Synthesizing with Multiplexers
  - Shannon's Expansion Theorem
- **Decoder**
  - Decoder with Enable Input
  - Decoder Tree
  - Multiplexers with Decoder
  - Decoder as Demultiplexer
- **Encoder**
- **Code Converters**
- **Comparator**

Adders 加法器

- **Addition**

Decimal

$$\begin{array}{rrr} & 2 & 2 & 5 \\ + & 3 & 6 & 3 \\ \hline & 5 & 8 & 8 \end{array} \qquad \begin{array}{rrr} & 5 & 8 & 8 \\ + & 4_{1} & 6_{1} & 3 \\ \hline 1 & 0 & 5 & 1 \end{array}$$

x	y	Carry 进位 c	Sum 和 s
5	3	0	8
8	3	1	1

Binary

$$\begin{array}{rrr} & 1 & 0 & 0 \\ + & 0 & 1 & 1 \\ \hline & 1 & 1 & 1 \end{array} \qquad \begin{array}{rrr} & 1 & 1 & 1 \\ + & 1_{1} & 0_{1} & 1 \\ \hline 1 & 1 & 0 & 0 \end{array}$$

x	y	c	s
0	1	0	1
1	1	1	0

Adders

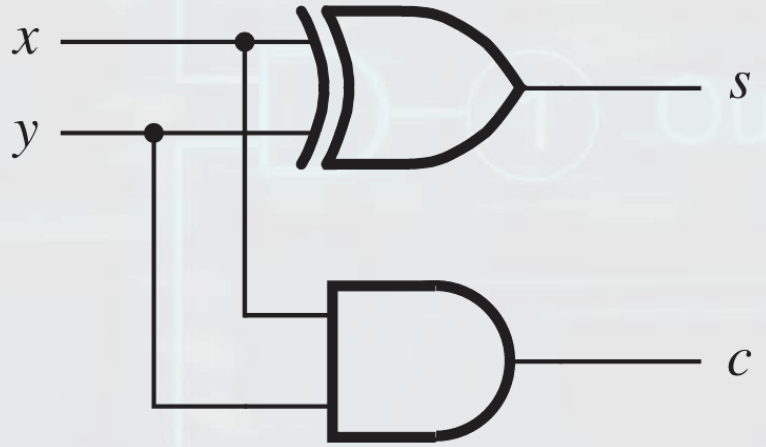
- **Half-adder (半加器)**
  Half-adder implements the addition of only two bits

x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$s = x\bar{y} + \bar{x}y = x \oplus y$$

$$c = x \cdot y$$

Circuit:



Graphical symbol:



Adders

- **Full-adder (全加器)**
  Full-adder considers carry-in from less significant bit

$$\begin{array}{rccc} & 5 & 8 & 8 \\ + & 4_1 & 6_1 & 3 \\ \hline & 1 & 0 & 5 & 1 \end{array}$$

$$\begin{array}{rccc} & 1 & 1 & 1 \\ + & 1_1 & 0_1 & 1 \\ \hline & 1 & 1 & 0 & 0 \end{array}$$

x_i	y_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

c_i : carry-in 来自低位的进位

c_{i+1} : carry-out 本位向高位的进位

Adders

- **Full-adder**

x_i	y_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

x_i \ $y_i c_i$	00	01	11	10
0		1		1
1	1		1	

$$s = \bar{x}_i \bar{y}_i c_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i$$

x_i \ $y_i c_i$	00	01	11	10
0			1	
1		1	1	1

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

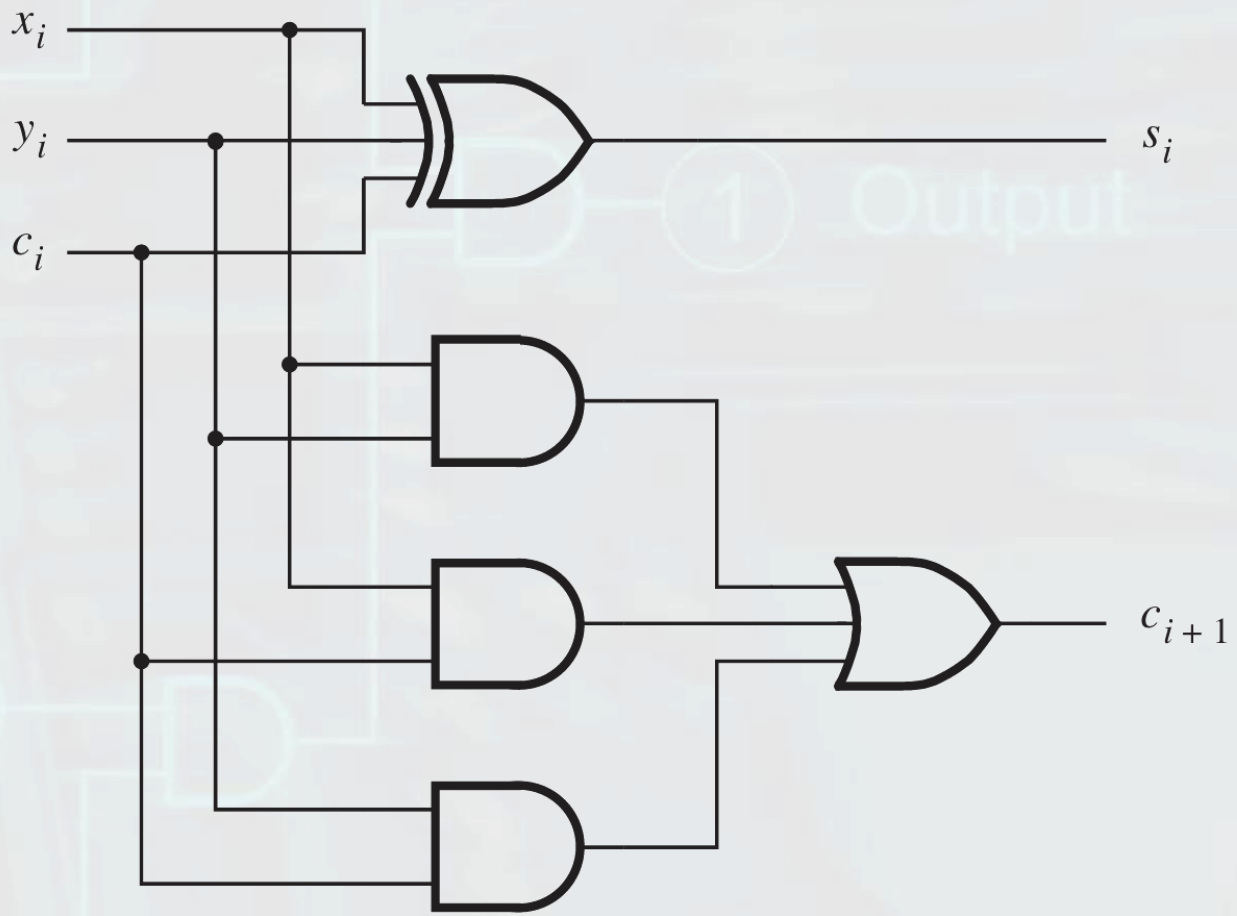
Adders

- **Full-adder**

$$
\begin{aligned}
s_i &= \bar{x}_i \bar{y}_i c_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i \\
&= \left(\bar{x}_i y_i \bar{c} + x_i \bar{y}_i \bar{c}_i\right) + \left(\bar{x}_i \bar{y}_i c_i + x_i y_i c_i\right) \\
&= \left(\bar{x}_i y_i + x_i \bar{y}_i\right)\bar{c}_i + \left(\bar{x}_i \bar{y}_i + x_i y_i\right) c_i \\
&= \left(x_i \oplus y_i\right)\bar{c}_i + \left(x_i \odot y_i\right) c_i \\
&= \left(x_i \oplus y_i\right)\bar{c}_i + \left(\overline{x_i \oplus y_i}\right) c_i \\
&= x_i \oplus y_i \oplus c_i
\end{aligned}
$$

$$
c_{i+1} = x_i y_i + x_i c_i + y_i c_i
$$





Adders

- **Full-adder with Two Half-adders**

$$x_i + y_i + c_i = (x_i + y_i) + c_i$$

$$\begin{array}{r} {\scriptstyle 1}\;\;{\scriptstyle 1} \\ +\quad 1\;1\;1 \\ \hline 1\;1\;0 \end{array} \qquad \begin{array}{r} \;\;1 \\ +\quad 1 \\ \hline 1\;0 \\ +\quad 1 \\ \hline 1\;1 \end{array} \;{\scriptstyle 1}$$

comes from the carry of $x_i + y_i$

$$\begin{array}{r} {\scriptstyle 1}\;\;{\scriptstyle 1} \\ +\quad 1\;0\;1 \\ \hline 1\;0\;0 \end{array} \qquad \begin{array}{r} \;\;1 \\ +\quad 0 \\ \hline 1 \\ +\quad 1 \\ \hline 1\;0 \end{array} \;{\scriptstyle 1}$$

comes from the carry of $s_{x_i + y_i} + c_i$

Adders

- **Full-adder with Two Half-adders**

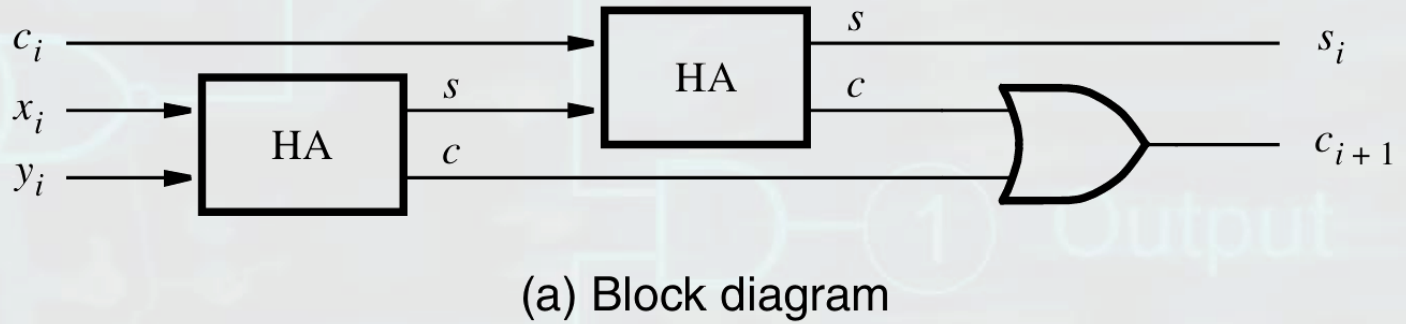
$$x_i + y_i + c_i = (x_i + y_i) + c_i$$

$$\begin{array}{rr} & x_i \\ + & y_i \\ \hline & c_1\ s_1 \end{array}$$ 1st half-adder

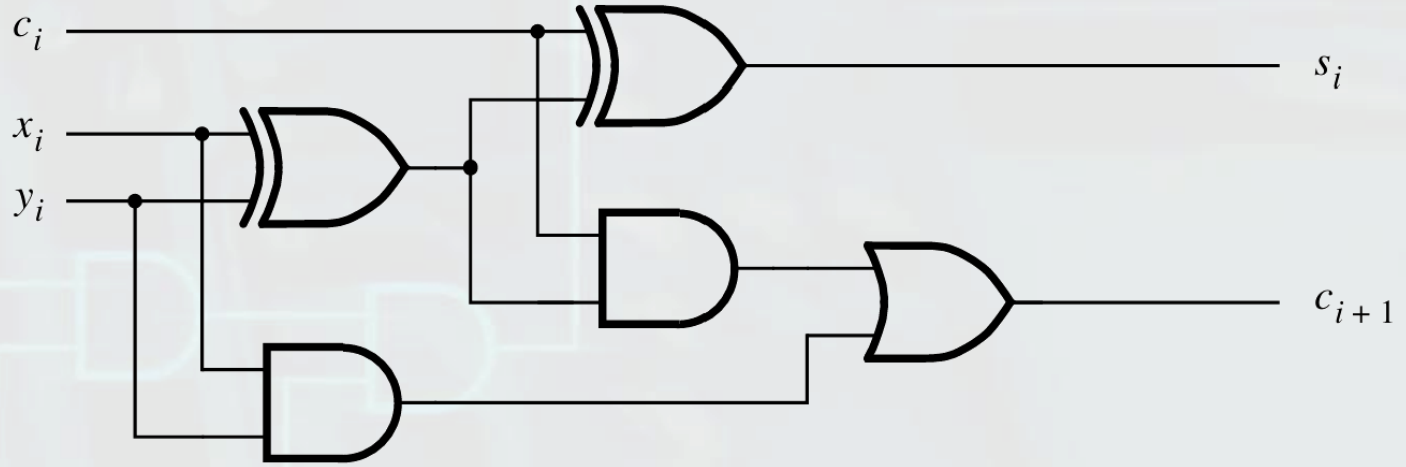
$$\begin{array}{rr} & s_1 \\ + & c_i \\ \hline & c_2\ s_i \end{array}$$ 2nd half-adder

$c_{i+1} = c_1 + c_2$

either c_1 or c_2 is "1"



(a) Block diagram



(b) Detailed diagram

Adders

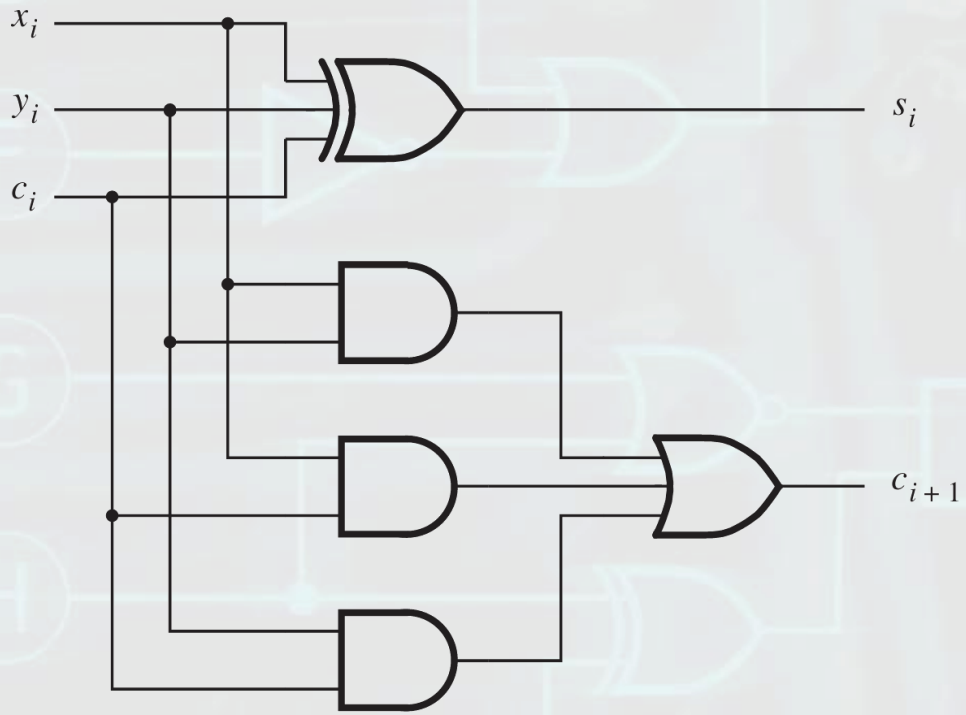
- **Another Form of Full-adder**

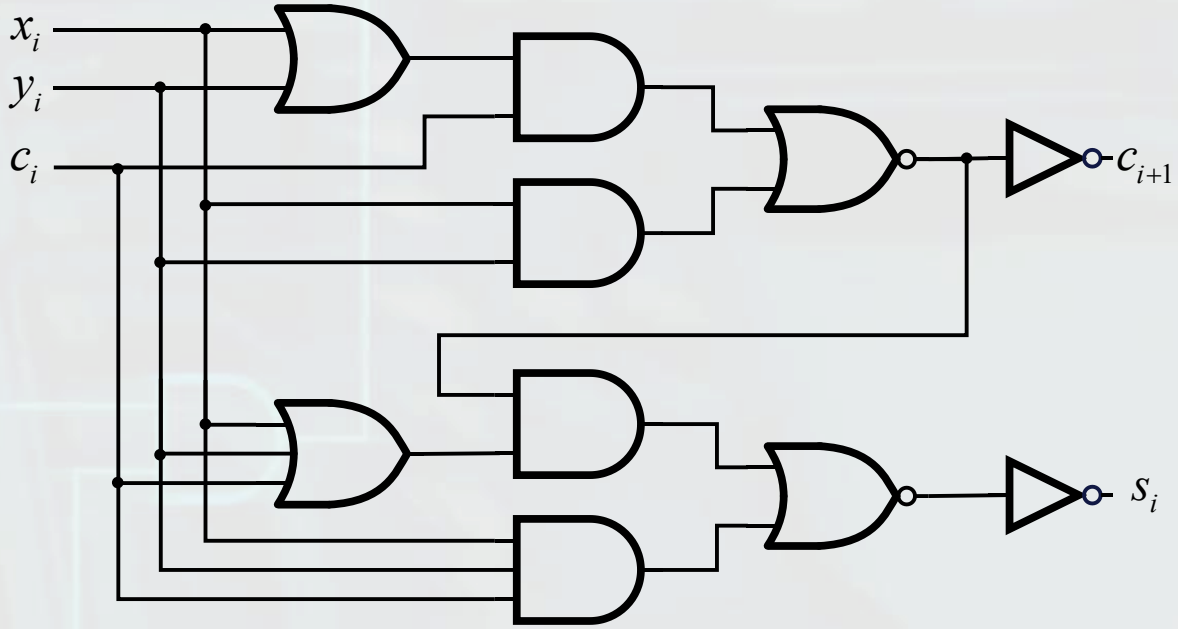
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

$$s_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i y_i + (x_i + y_i) c_i$$

$$s_i = \overline{c}_{i+1} (x_i + y_i + c_i) + x_i y_i c_i$$





Verilog Vode of Full-adder

```
module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output s, Cout;

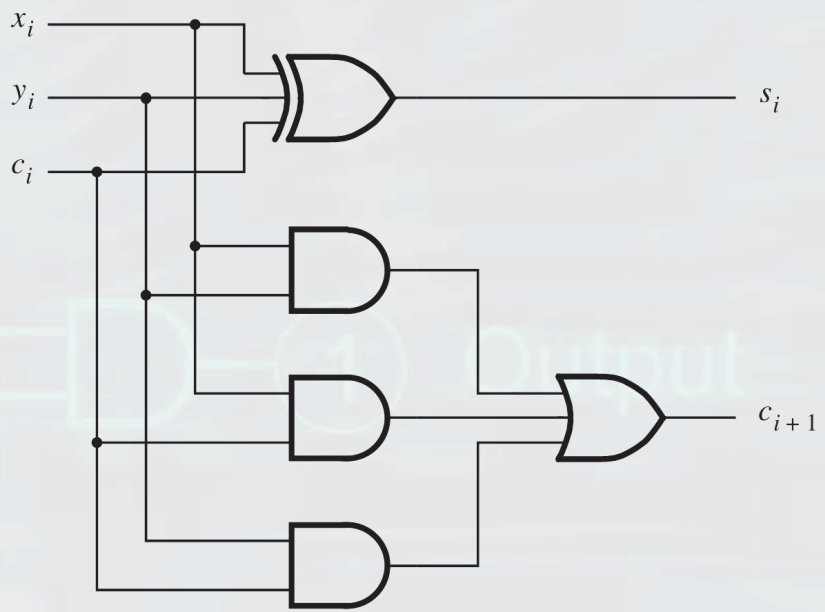
    xor (s, x, y, Cin);
    and (z1, x, y);
    and (z2, x, Cin);
    and (z3, y, Cin);
    or (Cout, z1, z2, z3);

endmodule
```

```
module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output s, Cout;

    xor (s, x, y, Cin);
    and (z1, x, y),
        (z2, x, Cin),
        (z3, y, Cin);
    or (Cout, z1, z2, z3);

endmodule
```




```
module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output s, Cout;

    assign s = x y Cin;
    assign Cout = (x & y) | (x & Cin) | (y & Cin);

endmodule
```

```
module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output s, Cout;

    assign s = x y Cin,
           Cout = (x & y) | (x & Cin) | (y & Cin);

endmodule
```

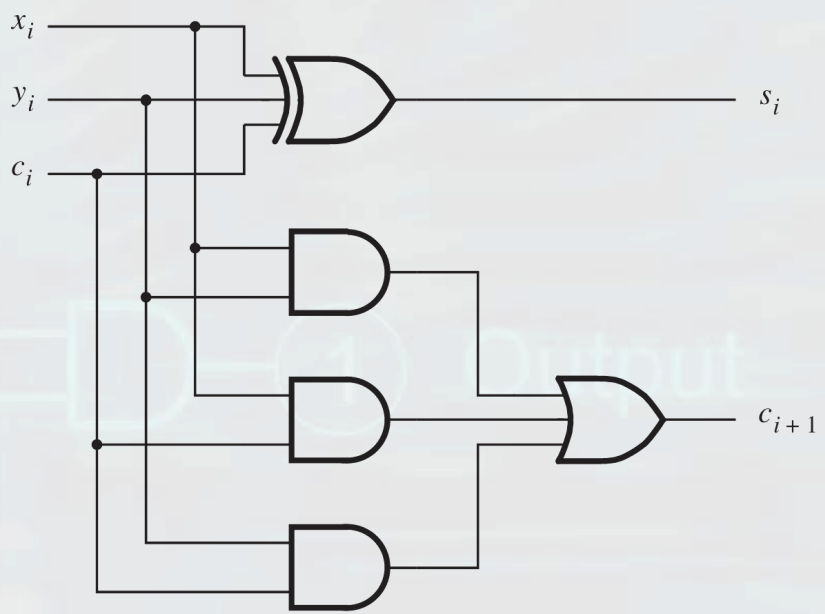
Verilog Vode of Full-adder

Behavioral description

```
module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output reg s, Cout;

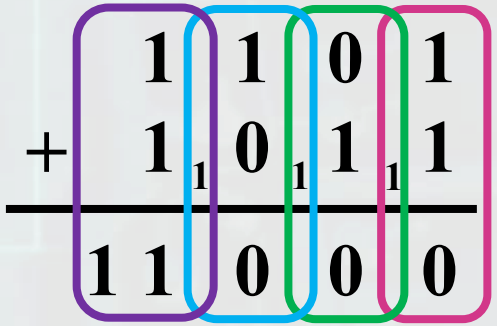
    always @(x, y, Cin)
        {Cout, s} = x + y + Cin;

endmodule
```



Adders

- **Ripple-Carry Adder (行波进位加法器)**
  - We do addition of multiple bits from the least-significant digit to the most-significant digit

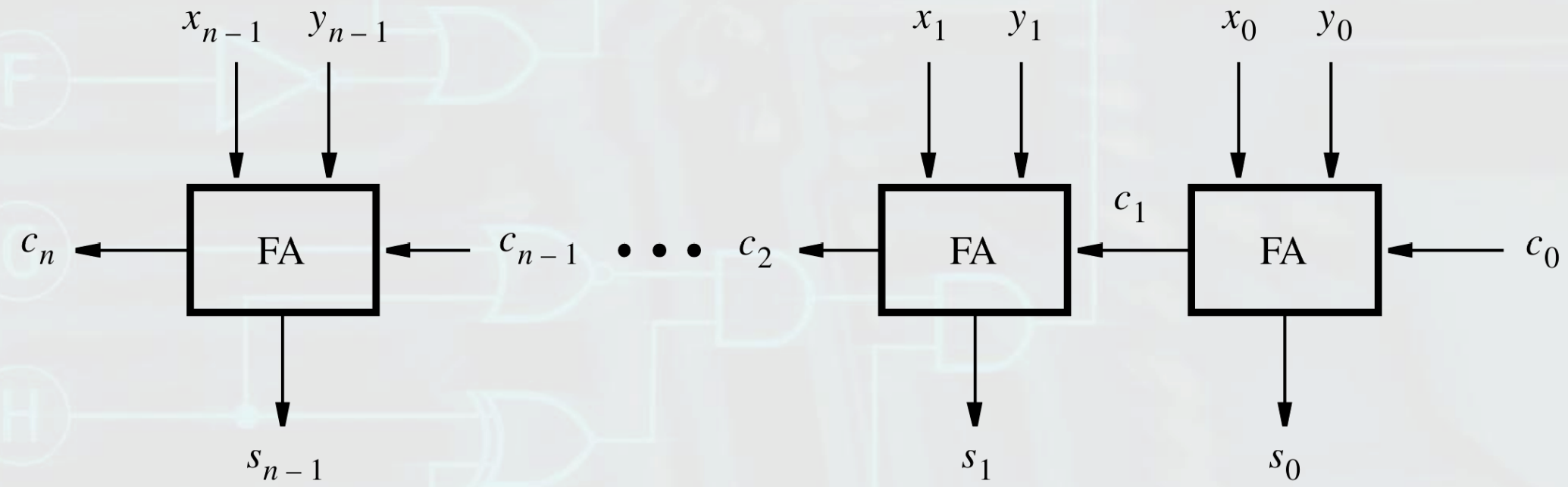


  - Carry produced in position i is added to the operands in position $i+1$

Adders

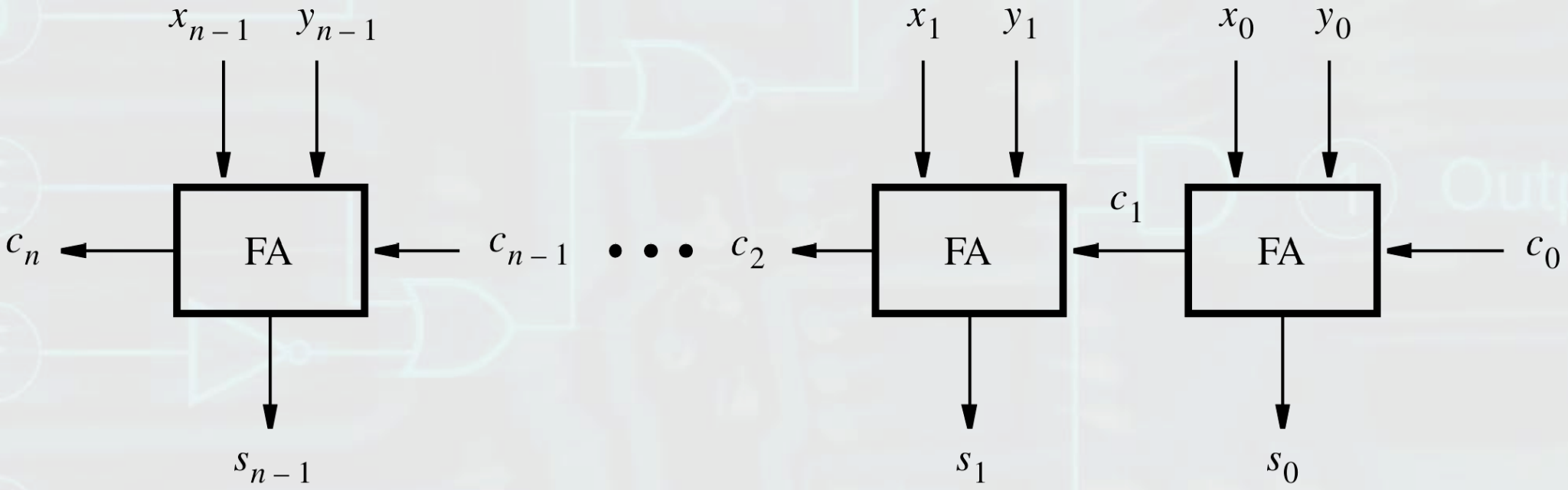
- **Ripple-Carry Adder (行波进位加法器)**
  - Carry produced in position i is added to the operands in position $i+1$, carry signals "**ripple**" through the full-adder stages

$$X + Y = x_{n-1}x_{n-2}\cdots x_1x_0 + y_{n-1}y_{n-2}\cdots y_1y_0$$

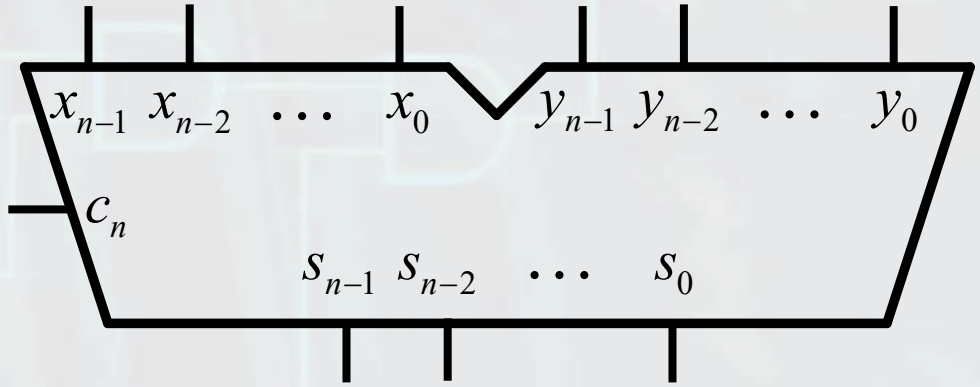


Adders

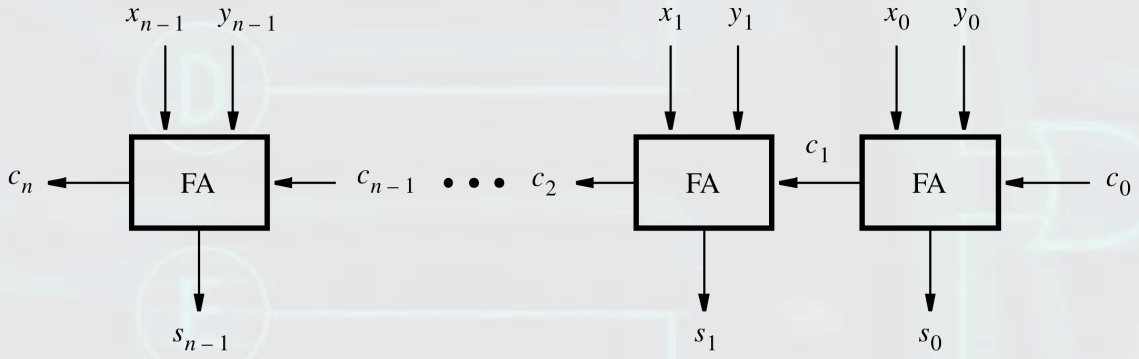
- **Ripple-Carry Adder (行波进位加法器)**



Graphical symbol:



Verilog Vode of Ripple-Carry Adder




```
module adder4 (carryin, X, Y, S, carryout);
    input carryin;
    input [3:0] X, Y;
    output [3:0] S;
    output carryout;
    wire [3:1] C;

    fulladd stage0 (carryin, X[0], Y[0], S[0], C[1]);
    fulladd stage1 (C[1], X[1], Y[1], S[1], C[2]);
    fulladd stage2 (C[2], X[2], Y[2], S[2], C[3]);
    fulladd stage3 (C[3], X[3], Y[3], S[3], carryout);

endmodule
```

A 4-bit adder

```
module addern (carryin, X, Y, S, carryout);
    parameter n = 32;
    input carryin;
    input [n –1:0] X, Y;
    output [n –1:0] S;
    output carryout;
    wire [n:0] C;

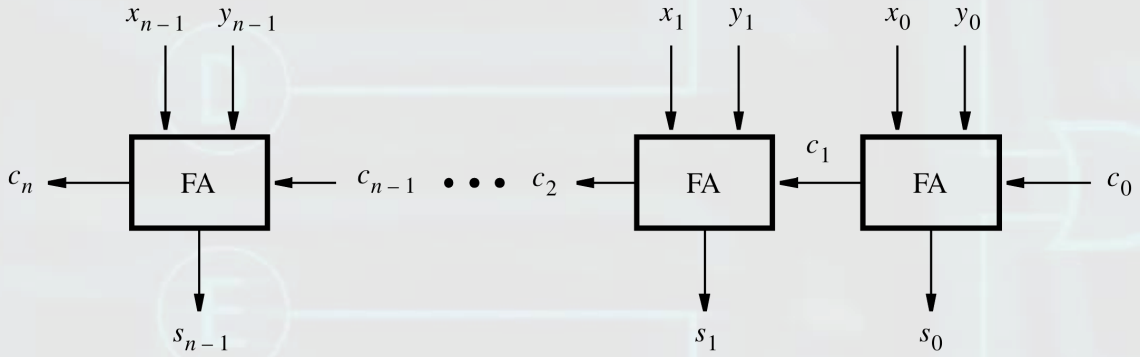
    assign C[0] = carryin;
    assign carryout = C[n];

    genvar i;
    generate
        for (i = 0; i < = n –1; i = i+1)
        begin:addbit
            fulladd stage (C[i], X[i], Y[i], S[i], C[i+1]);
        end
    endgenerate

endmodule
```

A 32-bit adder
using the **generate** statement

Verilog Vode of Ripple-Carry Adder




```
module addern (carryin, X, Y, S, carryout);
    parameter n = 32;
    input carryin;
    input [n– 1:0] X, Y;
    output reg [n –1:0] S;
    output reg carryout;
    reg [n:0] C;
    integer k;

    always @(X, Y, carryin)
    begin
      C[0] = carryin;
      for (k = 0; k < n; k = k+1)
      begin
        S[k] = X[k] ^ Y[k] ^ C[k];
        C[k+1] = (X[k] & Y[k]) | (X[k] & C[k]) (Y[k] & C[k]);
      end
      carryout = C[n];
    end

endmodule
```

A 32-bit adder
Logical expression description

Verilog Vode of Ripple-Carry Adder

```
module addern (carryin, X, Y, S, carryout, overflow);
    parameter n = 32;
    input carryin;
    input [n– 1:0] X, Y;
    output reg [n– 1:0] S;
    output reg carryout, overflow;

    always @(X, Y, carryin)
    begin
      {carryout, S} = X + Y + carryin;
      overflow = (X[n– 1] & Y[n– 1] & S[n– 1]) | ( X[n– 1] & Y[n– 1] & S[n– 1]);
    end

endmodule
```

A 32-bit adder
using arithmetic assignment

Practice:

Problem: $A = a_7 a_6 \cdots a_1 a_0$ an eight-bit unsigned number, $P = p_9 p_8 \cdots p_1 p_0$ is the product of A multiplying by 3 ($P = 3A$). Design a circuit to generate P.

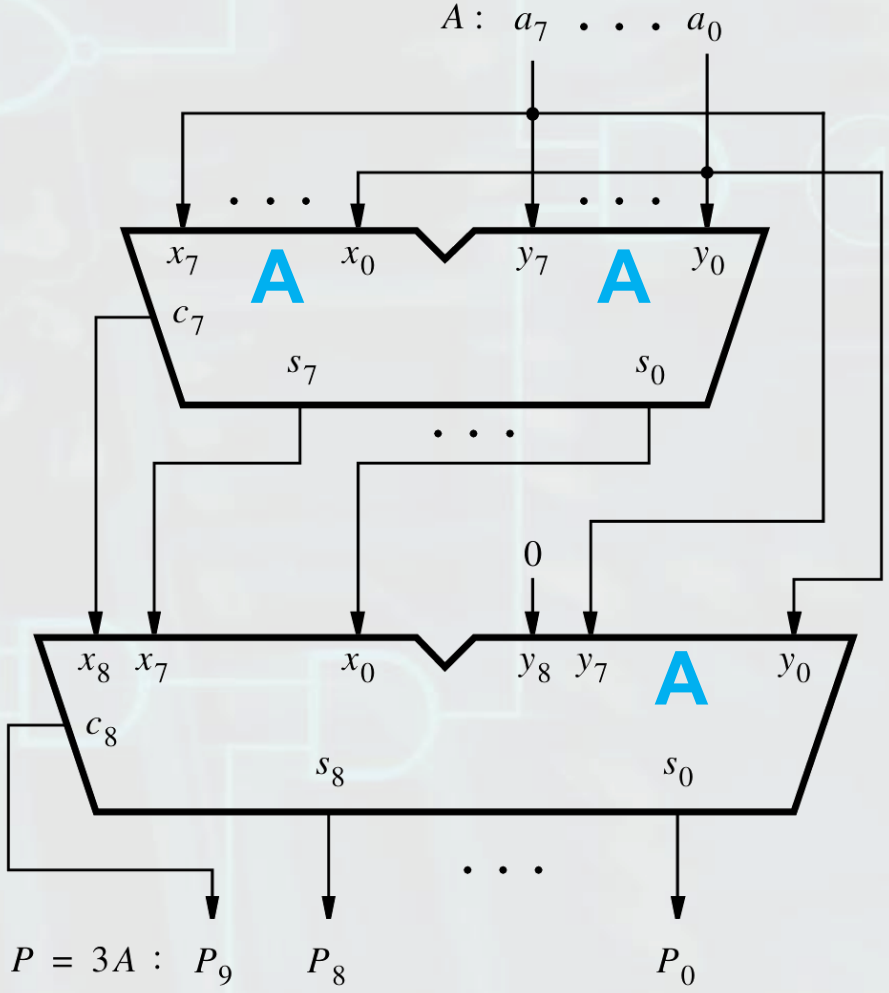
Solution:

① $P = A + A + A$

8 bits

9 bits

10 bits



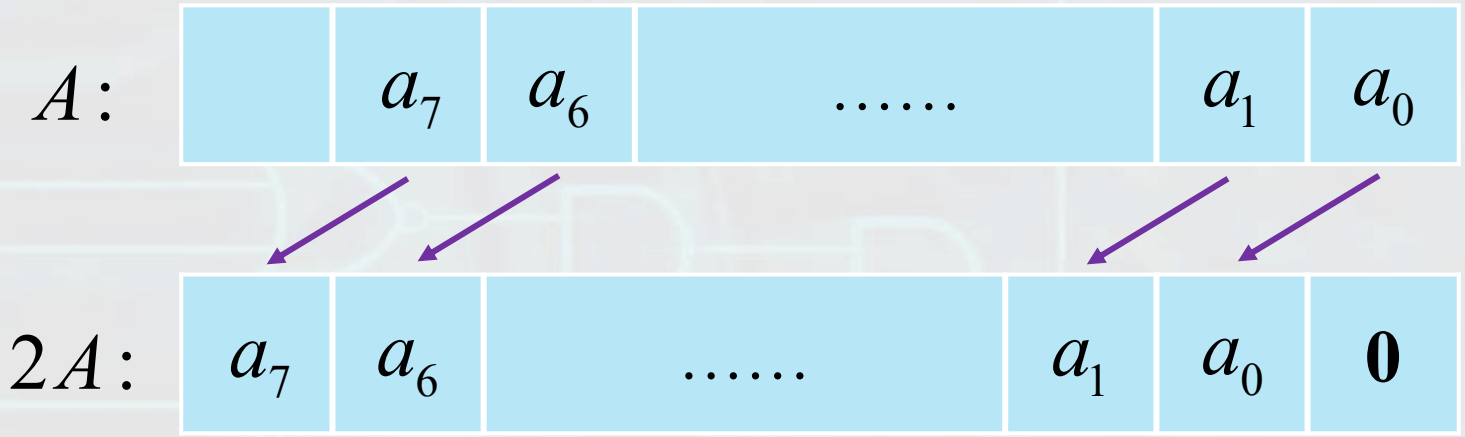
Practice:

Problem: $A = a_7 a_6 \cdots a_1 a_0$ an eight-bit unsigned number, $P = p_9 p_8 \cdots p_1 p_0$ is the product of A multiplying by 3 ($P = 3A$). Design a circuit to generate P.

Solution:

② $P = 2A + A$

"**×2**" can be generated by shifting one bit to the left

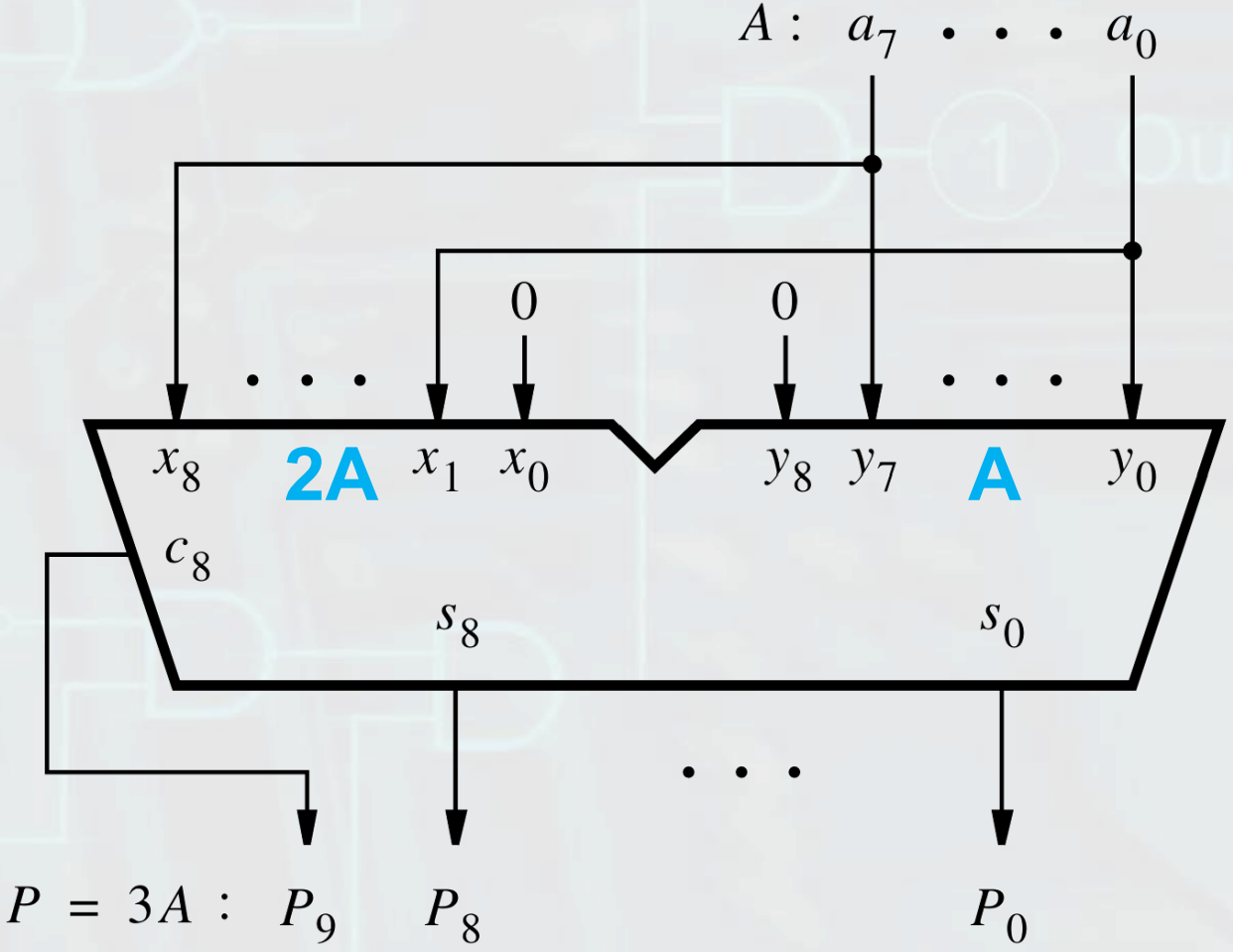


Practice:

Problem: $A = a_7 a_6 \cdots a_1 a_0$ an eight-bit unsigned number, $P = p_9 p_8 \cdots p_1 p_0$ is the product of A multiplying by 3 ($P = 3A$). Design a circuit to generate P.

Solution:

② $P = \underbrace{2A}_{\text{9 bits}} + \underbrace{A}_{\text{10 bits}}$



Adder/Subtractor Unit

- **Adder/Subtractor Unit (加法器/减法器单元)**
- Common unit for addition and subtraction

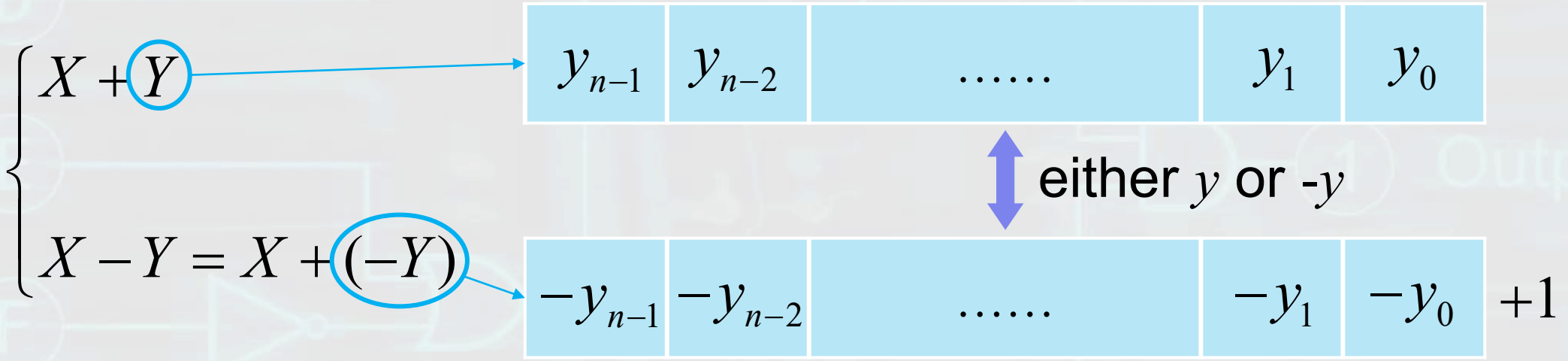
$$X - Y = X + (-Y)$$

use complement 补码

Y:	y_{n-1}	y_{n-2}	……	y_1	y_0
1's complement of $-Y$ $K_1(-Y)$:	$-y_{n-1}$	$-y_{n-2}$	……	$-y_1$	$-y_0$
2's complement of $-Y$ $K_2(-Y) = K_1(-Y)+1$	$-y_{n-1}$	$-y_{n-2}$	……	$-y_1$	$-y_0$ $+1$

Adder/Subtractor Unit

- Common unit for addition and subtraction





$y \oplus 0 = y$

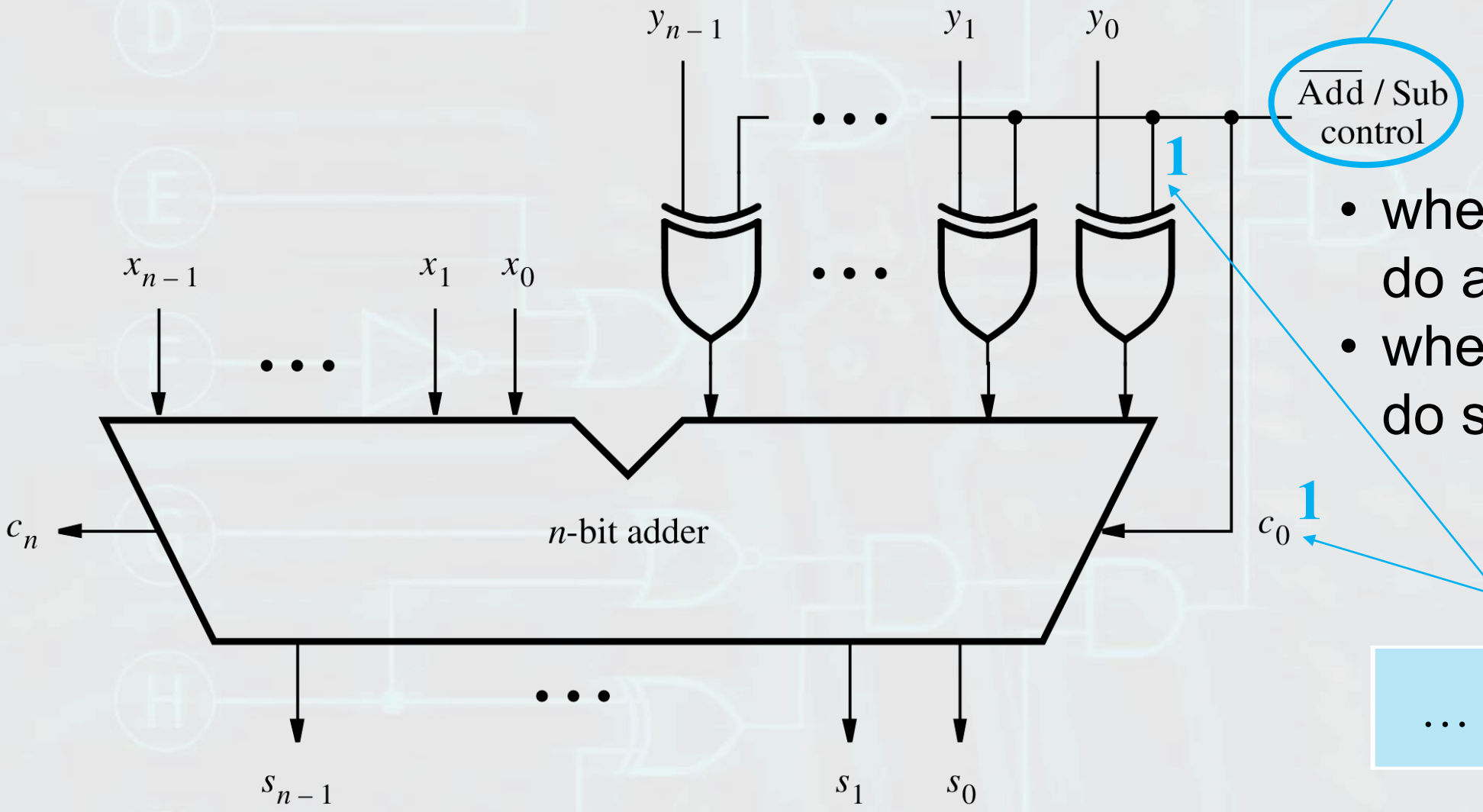


$y \oplus 1 = \bar{y}$

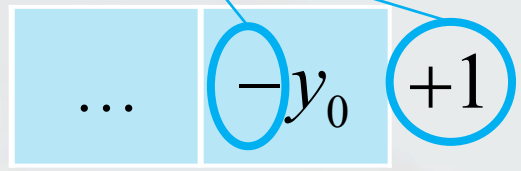
Adder/Subtractor Unit

A **bar** over the name (名字上的横线) means that the action is enabled if the control signal is "0"

- Common unit for addition and subtraction

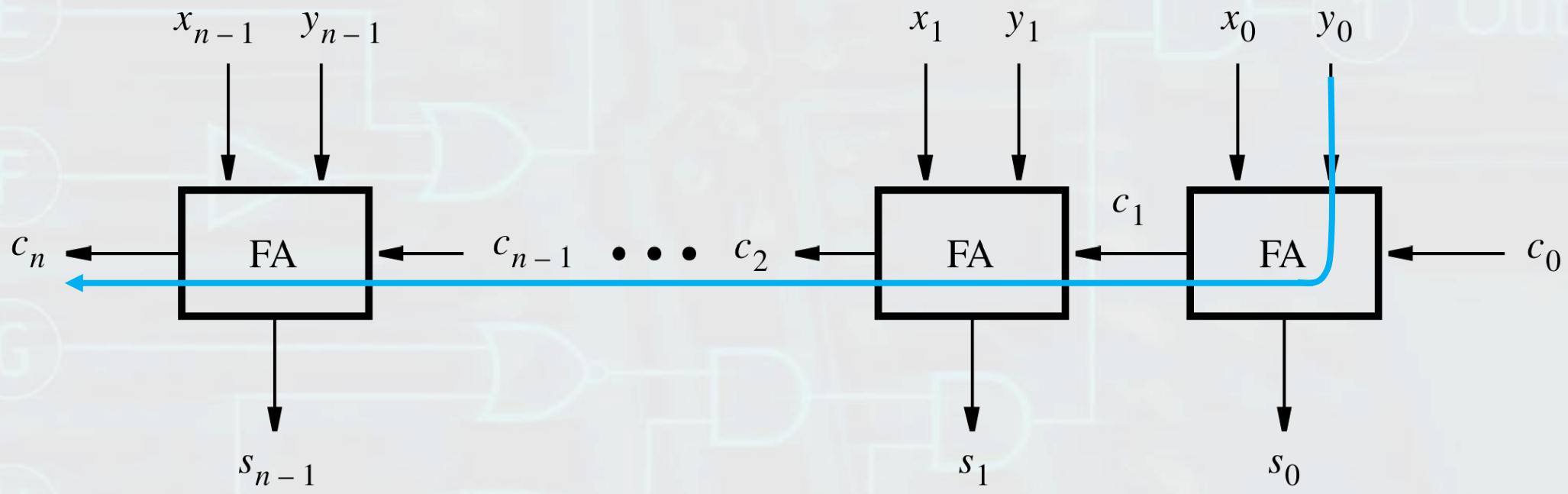


- when control = 0, do addition
- when control = 1, do subtraction



Delay of Ripple-Carry Adder

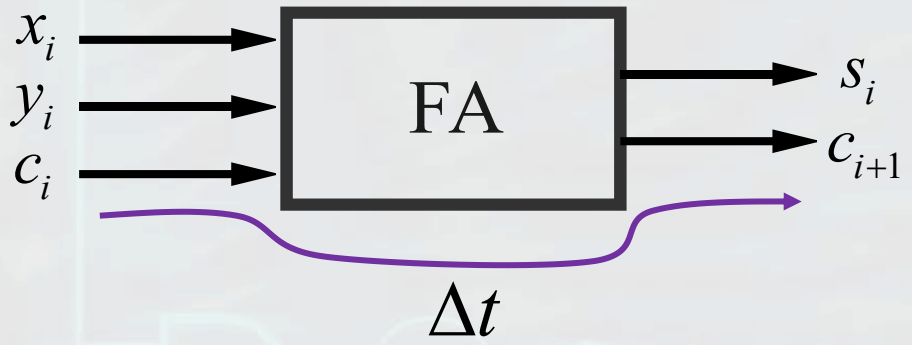
- The longest delay is often called the **critical-path delay** 关键路径延时
- The path that causes this delay is called the **critical path**

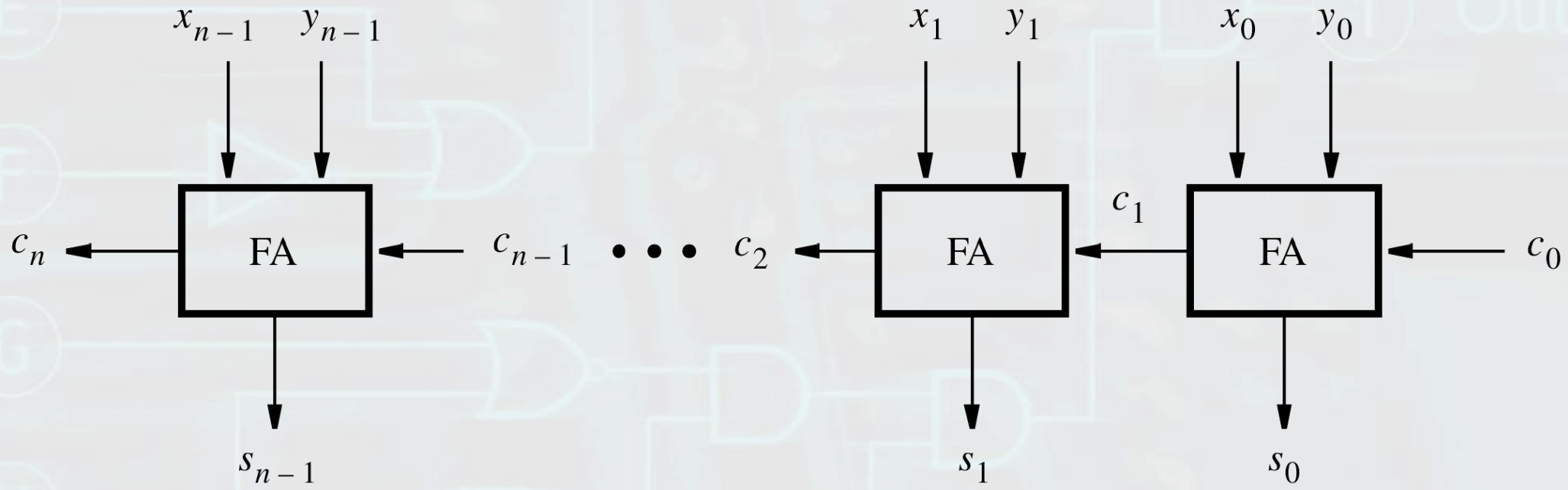


Delay of Ripple-Carry Adder

- Assume delay of a full-adder is Δt
- Delay of ripple-carry adder is:

$$\Delta t_{rc} = n \times \Delta t$$





Issue: If n is large (32/64bit), the delay may become unacceptably high!

Fast Adders

- **Carry-Lookahead Adder (超前进位加法器)**
  - Carry is the key step for critical-path delay
  - A possible quicker way to calculate carry?

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i = x_i y_i + (x_i + y_i) c_i$$

$$c_1 = x_0 y_0 + (x_0 + y_0) c_0$$
$$c_2 = x_1 y_1 + (x_1 + y_1) c_1 = x_1 y_1 + (x_1 + y_1)(x_0 y_0 + (x_0 + y_0) c_0)$$
$$c_3 = x_2 y_2 + (x_2 + y_2) c_2 = x_2 y_2 + (x_2 + y_2)(x_1 y_1 + (x_1 + y_1)(x_0 y_0 + (x_0 + y_0) c_0))$$
……

Actually, calculating c_{i+1} does **not need to wait** for the result of c_i

Carry-Lookahead Adder

- A simpler way for expression

$$\begin{cases} g_i = x_i y_i & \longrightarrow \text{generate function 生成函数} \\ p_i = x_i + y_i & \longrightarrow \text{propagate function 传播函数} \\ c_{i+1} = x_i y_i + (x_i + y_i) c_i = g_i + p_i c_i \end{cases}$$

$$c_{i+1} = g_i + p_i c_i = g_i + p_i (g_{i-1} + p_{i-1} c_{i-1}) = g_i + p_i g_{i-1} + p_i p_{i-1} c_{i-1}$$

$$\begin{cases} c_1 = g_0 + p_0 c_0 \\ c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0 \\ c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\ \quad \vdots \\ c_{n+1} = g_n + p_n g_{n-1} + p_n p_{n-1} g_{n-2} + \cdots + p_n p_{n-1} \cdots p_2 p_1 g_0 + p_n p_{n-1} \cdots p_1 p_0 c_0 \end{cases}$$

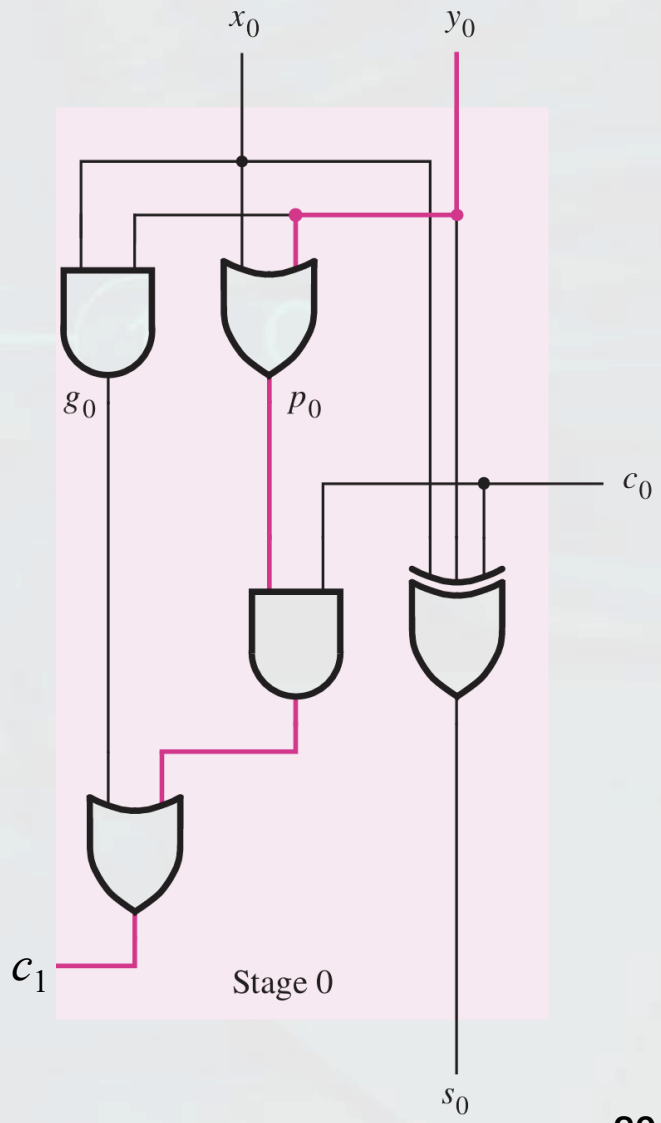
Carry-Lookahead Adder

- Delay of one stage of adder
  - Assume same delay d_{gate} for all gates
    假设所有的门都有同样的门延迟

$$\begin{cases} g_i = x_i y_i & \longrightarrow \text{one AND gate: } d_{gate} \\ p_i = x_i + y_i & \longrightarrow \text{one OR gate: } d_{gate} \\ c_{i+1} = g_i + p_i c_i \end{cases}$$

one AND gate + one OR gate: $2d_{gate}$

So delay from x_i , y_i to c_{i+1} is $3d_{gate}$



Carry-Lookahead Adder

- Delay of **ripple-carry adder**

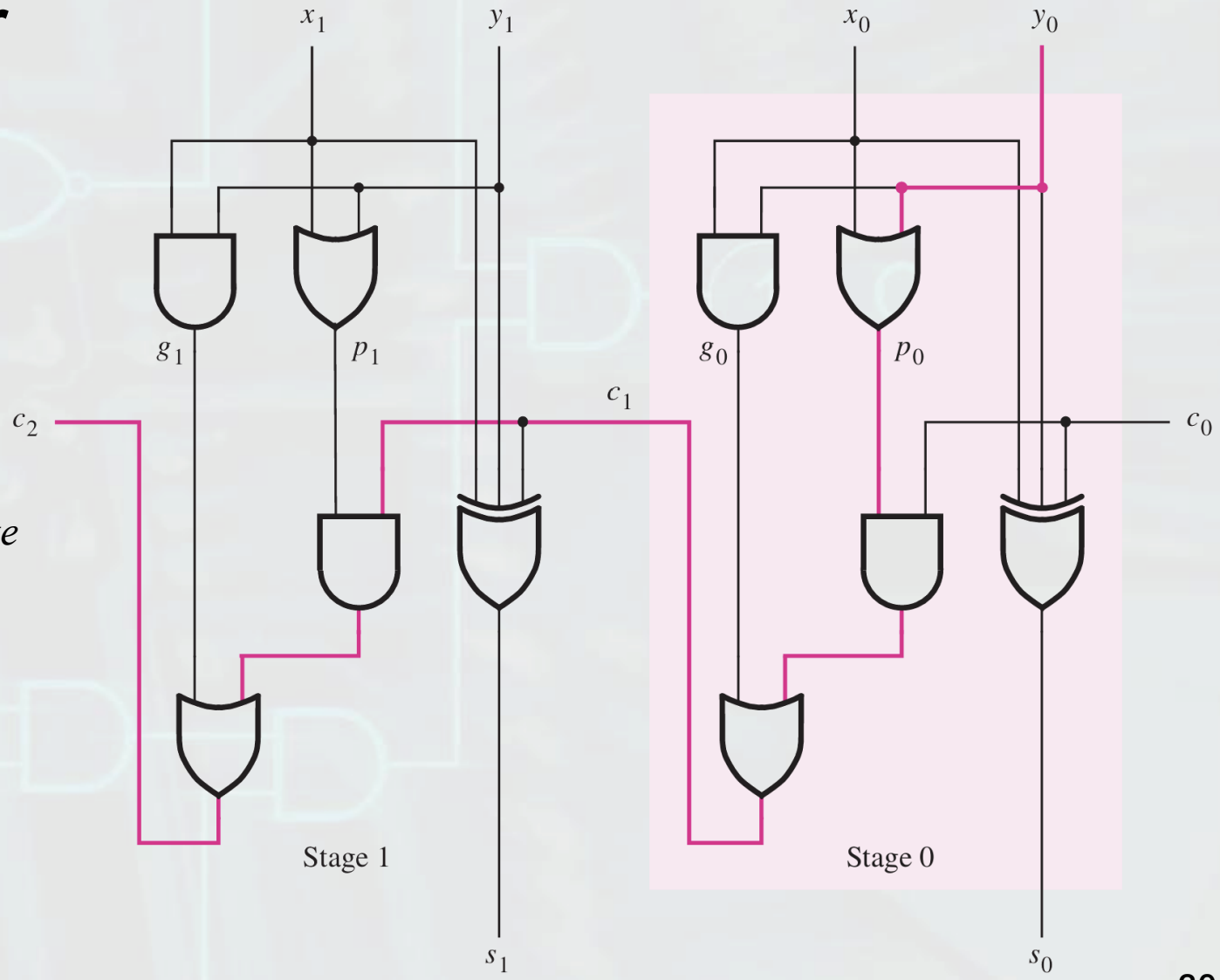
$$\begin{cases} g_i = x_i y_i \\ p_i = x_i + y_i \\ c_{i+1} = g_i + p_i c_i \end{cases}$$

Delay for the 1st stage: $d_{stage1} = 3d_{gate}$

Delay for other stages: $d = 2d_{gate}$

Delay for n-stage ripple-carry adder:

$$d_{ripple-carry} = (2n+1) \cdot d_{gate}$$



Carry-Lookahead Adder

- Delay of **carry-lookahead adder**

$c_{i+1} = g_i + p_i c_i$ → Delay from x_i , y_i to c_{i+1} is $3d_{gate}$

$$\begin{cases} c_1 = g_0 + p_0 c_0 \\ c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0 \\ \quad \vdots \\ c_{n+1} = g_n + p_n g_{n-1} + p_n p_{n-1} g_{n-2} + \cdots + p_n p_{n-1} \cdots p_2 p_1 g_0 + p_n p_{n-1} \cdots p_1 p_0 c_0 \end{cases}$$

If no limit for fan-in of gates,
delays for all carry are same: $3d_{gate}$

Carry-Lookahead Adder

- Delay of carry-lookahead adder

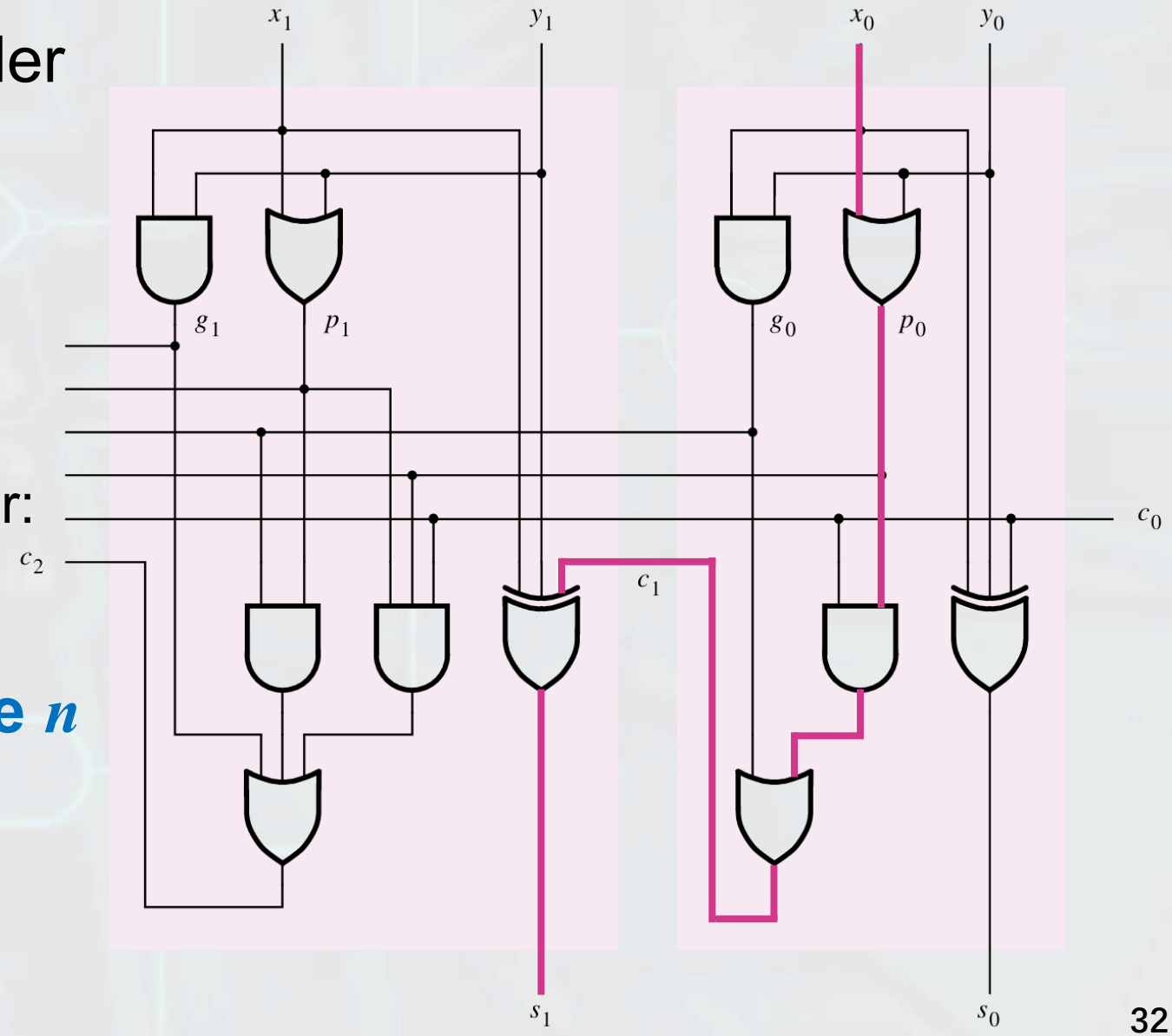
$$\begin{cases} g_i = x_i y_i \\ p_i = x_i + y_i \\ c_{i+1} = g_i + p_i c_i \end{cases}$$

Delay for n-stage carry-lookahead adder:

$$d_{carry-lookahead} = 4d_{gate}$$

faster for large n

$$d_{ripple-carry} = (2n+1) \cdot d_{gate}$$



Carry-Lookahead Adder

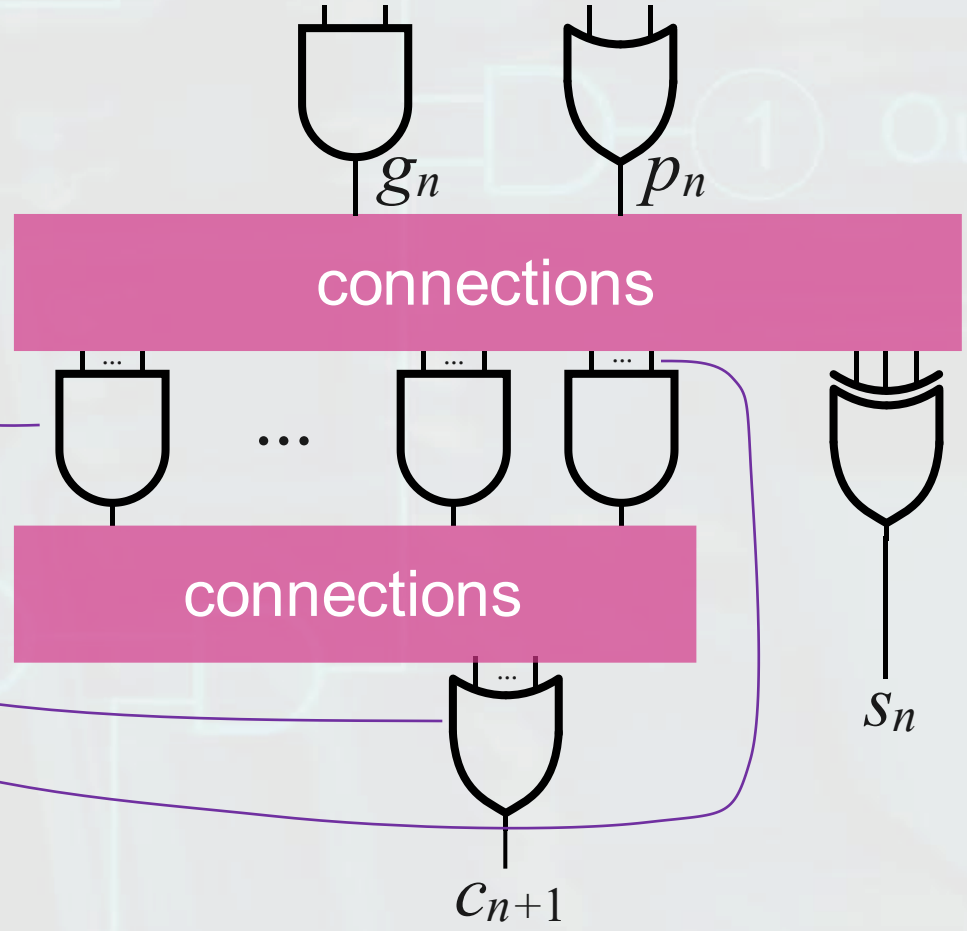
- **Issue** of carry-lookahead adder: **complexity**

$$c_{n+1} = g_n + p_n g_{n-1} + p_n p_{n-1} g_{n-2} + \cdots + p_n p_{n-1} \cdots p_2 p_1 g_0 + p_n p_{n-1} \cdots p_1 p_0 c_0$$

If $n = 32$:

- 32 AND gates for last stage
- 33 inputs for AND / OR gate

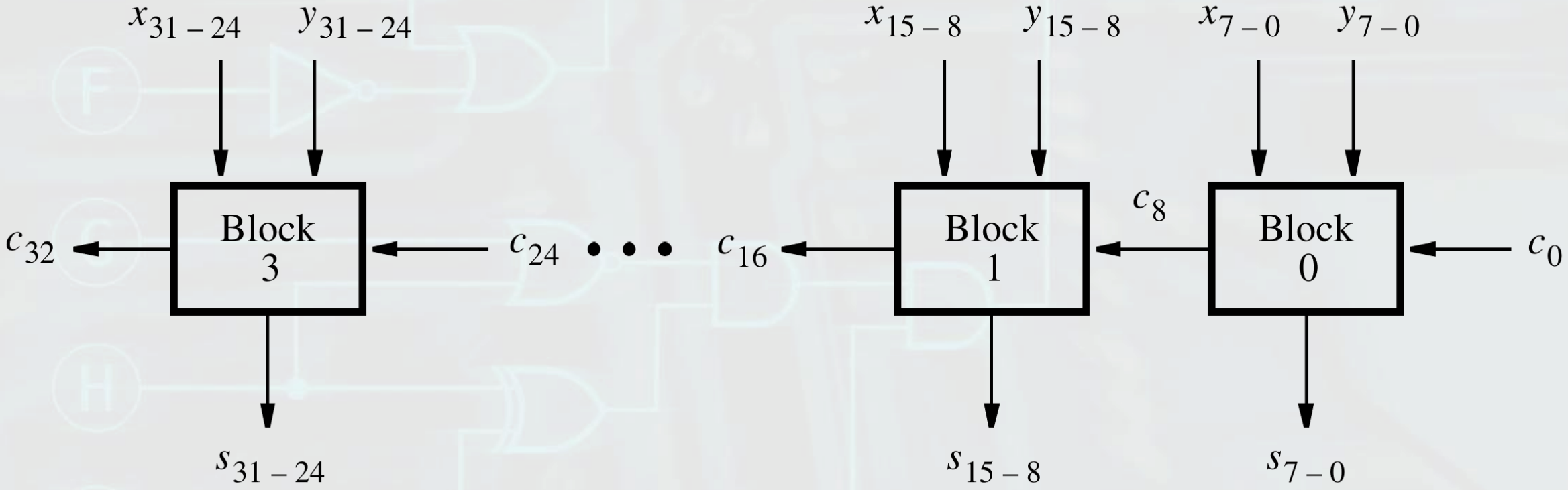
If $n = 64$?



Hierarchical Design for Large Adder

- **Balance between speed and complexity**
  - Hierarchical: split larger adder into smaller blocks
- **Approach 1**: **carry-lookahead** approach **within** blocks
  ripple-carry approach **between** blocks

e.g. for a 32-bit adder, divide into 4 8-bit adder blocks



Hierarchical Design for Large Adder

- **Approach 2**: 2nd-level **carry-lookahead between** blocks

$$c_8 = g_7 + p_7 g_6 + p_7 p_6 g_5 + p_7 p_6 p_5 g_4 + p_7 p_6 p_5 p_4 g_3 + p_7 p_6 p_5 p_4 p_3 g_2 + p_7 p_6 p_5 p_4 p_3 p_2 g_1 + p_7 p_6 p_5 p_4 p_3 p_2 p_1 g_0 + p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0 c_0$$

G_0 P_0

$$c_8 = G_0 + P_0 c_0$$

$$c_{16} = G_1 + P_1 c_8 = G_1 + P_1 G_0 + P_1 P_0 c_0$$

$$c_{24} = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0$$

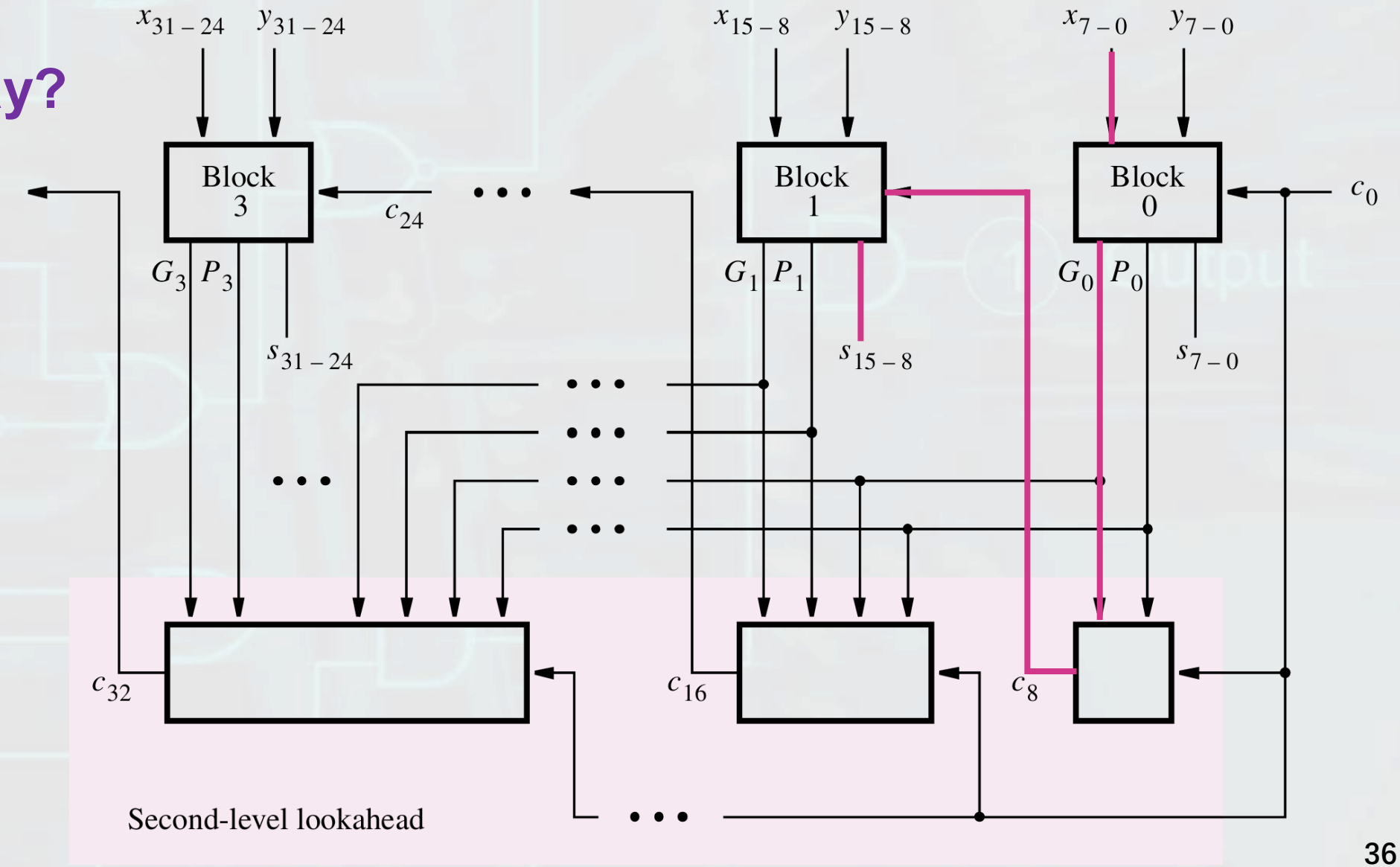
$$c_{32} = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0$$

Block does not need to wait for the carry from other blocks

Hierarchical Design for Large Adder

Critical path delay?

- G_0~G_3, P_0~P_3 are generated at the same delay
- c_8 is necessary to generate c_9~c_{16}



Hierarchical Design for Large Adder

- **Approach 2**: 2nd-level **carry-lookahead between** blocks
  - **Critical path delay**

$$p_i = x_i + y_i \longrightarrow x_i, y_i \rightarrow p_i, g_i \quad d_{gate}$$

$$G_0 = g_7 + p_7 g_6 + \cdots \longrightarrow p_i, g_i \rightarrow G_k \quad 2d_{gate}$$

$$+ p_7 p_6 p_5 p_4 p_3 p_2 p_1 g_0$$

$$P_0 = p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0 \longrightarrow p_i, g_i \rightarrow P_k \quad d_{gate}$$

$$c_8 = G_0 + P_0 c_0 \longrightarrow G_k, P_k \rightarrow c_{8,16,24,32} \quad 2d_{gate}$$

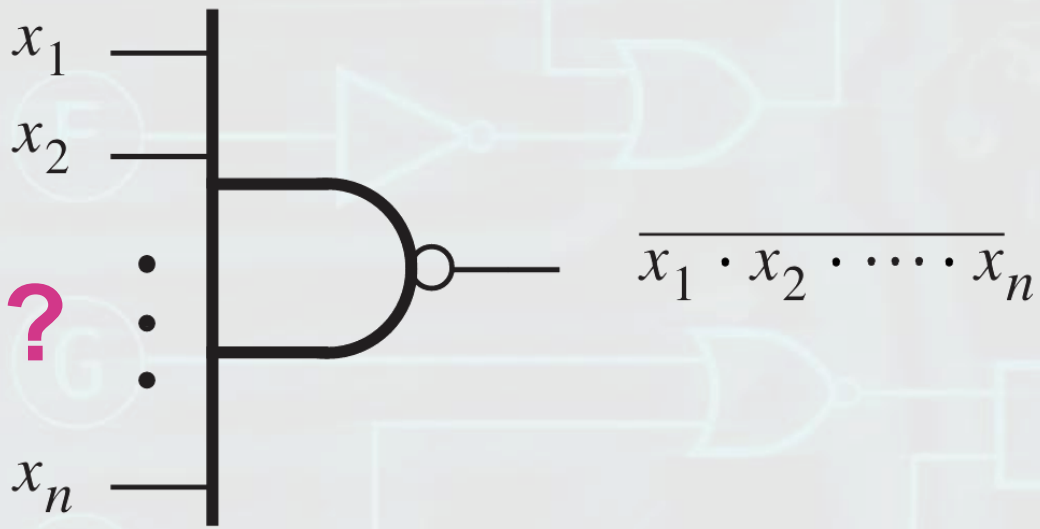
$$c_8 \rightarrow c_{9,10,\cdots,15} \rightarrow s_{9,10,\cdots,15}$$

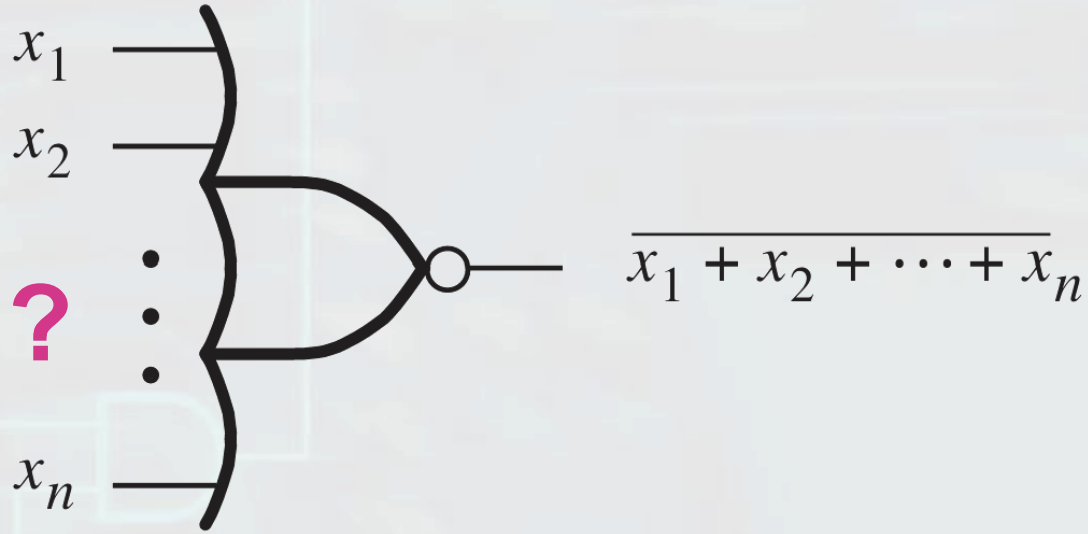
$2d_{gate}$ d_{gate}

critical path delay = $8d_{gate}$

Hierarchical Design for Large Adder

- **Approach 2**: 2nd-level **carry-lookahead between** blocks
  - **High fan-in issue**
  - fan-in: the number of gate inputs
  - In reality, **fan-in is limited** due to speed consideration





Hierarchical Design for Large Adder

- **Approach 2**: 2nd-level **carry-lookahead between** blocks
  - **High fan-in issue**
  - Suppose that the maximum fan-in of the gates is **four inputs**

$$c_8 = g_7 + p_7 g_6 + p_7 p_6 g_5 + p_7 p_6 p_5 g_4 + p_7 p_6 p_5 p_4 g_3 + p_7 p_6 p_5 p_4 p_3 g_2 + p_7 p_6 p_5 p_4 p_3 p_2 g_1 + p_7 p_6 p_5 p_4 p_3 p_2 p_1 g_0 + p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0 c_0$$

$3d_{gate}$

$$c_8 = g_7 + p_7 g_6 + p_7 p_6 g_5 + p_7 p_6 p_5 g_4 + (p_7 p_6 p_5 p_4)(p_3 p_2 p_1 g_0) c_0 + (p_7 p_6 p_5 p_4)(g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0)$$

$5d_{gate}$